



Materiały dydaktyczne

Przedmiot: Programowanie obiektowe/Pracownia programowania obiektowego

Opracował: Mirosław Miciak

Zadanie: Z2

Tematy: Podział programu na funkcje, Biblioteki języka programowania, Proste i złożone typy danych, Zmienne, Operacje na zmiennych: wejścia, wyjścia, arytmetyczne logiczne, Operacje na typach złożonych, Operatory arytmetyczne, przypisania, porównania, Operatory logiczne i bitowe, Operatory do obsługi łańcuchów, Instrukcja warunkowa i instrukcja wyboru, Instrukcje pętli, Wykorzystanie wybranej biblioteki matematycznej, Wykorzystanie bibliotek algorytmów, Funkcje, Klasa, obiekt, metoda, pole, Projektowanie prostych klas, Zmienne obiektowe, Zakres widoczności pól klasy, Metody klasy, Konstruktor i destruktor, Konstruktor kopiujący, Metody klasy - zakres widoczności, Prosta aplikacja obiektowa: konstruktor, destruktor, Składniki statyczne klasy, Klasy dziedziczone, Klasy zaprzyjaźnione, Dziedziczenie, Hermetyzacja, Polimorfizm, Funkcje zaprzyjaźnione z klasą, Metody do obsługi elementów statycznych klasy, Klasy bazowe i pochodne, Metody wirtualne, Klasy abstrakcyjne, Szablony klas dla prostych typów liczbowych, Obsługa wyjątków, Instrukcja throw, Obsługa błędów wykonania aplikacji..

Operacje na typach złożonych

Typy złożone w C# obejmują klasy i struktury, które mogą zawierać różne elementy, takie jak pola, właściwości, metody, zdarzenia i inne. Klasy są typami referencyjnymi, co oznacza, że obiekty klas są przechowywane na stercie, a zmienne przechowują referencje do tych obiektów. Struktury są typami wartościowymi, co oznacza, że są przechowywane bezpośrednio w zmiennych.

Klasy

Klasy są podstawowym budulcem programowania obiektowego w C#. Mogą zawierać pola (zmienne), właściwości (getter i setter), metody (funkcje) oraz zdarzenia. Klasy mogą dziedziczyć po innych klasach, co pozwala na tworzenie hierarchii klas i ponowne wykorzystanie kodu.

Przykład:

```
public class Person
{
    public string Name { get; set; }
    public int Age { get; set; }

    public void Introduce()
    {
        Console.WriteLine($"Hi, I'm {Name} and I'm {Age} years old.");
    }
}
```

Struktury

Struktury są podobne do klas, ale są typami wartościowymi. Są używane do reprezentowania małych, prostych obiektów, które nie wymagają dziedziczenia.

Przykład:

```
public struct Point
{
    public int X { get; set; }
    public int Y { get; set; }

    public Point(int x, int y)
    {
        X = x;
        Y = y;
    }
}
```

Operatory arytmetyczne

Operatory arytmetyczne są używane do wykonywania podstawowych operacji matematycznych. W C# dostępne są następujące operatory arytmetyczne:

- + (dodawanie)
- (odejmowanie)
- * (mnożenie)
- / (dzielenie)
- % (modulo)

Przykład:

```
int a = 10;
int b = 3;
Console.WriteLine(a + b); // Output: 13
Console.WriteLine(a - b); // Output: 7
Console.WriteLine(a * b); // Output: 30
Console.WriteLine(a / b); // Output: 3
Console.WriteLine(a % b); // Output: 1
```

Operatory przypisania

Operatory przypisania są używane do przypisywania wartości zmiennym. Oprócz podstawowego operatora przypisania =, C# oferuje również skrócone formy operatorów przypisania, które łączą operację arytmetyczną z przypisaniem:

- += (dodawanie i przypisanie)
- = (odejmowanie i przypisanie)
- *= (mnożenie i przypisanie)
- /= (dzielenie i przypisanie)
- %= (modulo i przypisanie)

Przykład:

```
int c = 5;
c += 3; // c = c + 3
Console.WriteLine(c); // Output: 8
c *= 2; // c = c * 2
Console.WriteLine(c); // Output: 16
```

Operatory porównania

Operatory porównania są używane do porównywania wartości. Wynikiem operacji porównania jest wartość logiczna true lub false. W C# dostępne są następujące operatory porównania:

== (równość)

!= (nierówność)

< (mniejszy niż)

> (większy niż)

<= (mniejszy lub równy)

>= (większy lub równy)

Przykład:

```
int d = 10;
int e = 20;
Console.WriteLine(d == e); // Output: False
Console.WriteLine(d != e); // Output: True
Console.WriteLine(d < e); // Output: True
Console.WriteLine(d > e); // Output: False
```

Operatory logiczne

Operatory logiczne są używane do wykonywania operacji logicznych na wartościach boolowskich. W C# dostępne są następujące operatory logiczne:

&& (AND logiczne)

|| (OR logiczne)

! (NOT logiczne)

Przykład:

```
bool f = true;
bool g = false;
Console.WriteLine(f && g); // Output: False
Console.WriteLine(f || g); // Output: True
Console.WriteLine(!f); // Output: False
```

Operatory bitowe

Operatory bitowe są używane do manipulowania poszczególnymi bitami wartości liczbowych. W C# dostępne są następujące operatory bitowe:

& (AND bitowe)

| (OR bitowe)

^ (XOR bitowe)

~ (NOT bitowe)

<< (przesunięcie bitowe w lewo)

>> (przesunięcie bitowe w prawo)

Przykład:

```
int h = 6; // 110 binarnie
int i = 3; // 011 binarnie
Console.WriteLine(h & i); // Output: 2 (010 binarnie)
Console.WriteLine(h | i); // Output: 7 (111 binarnie)
Console.WriteLine(h ^ i); // Output: 5 (101 binarnie)
Console.WriteLine(~h); // Output: -7 (inverts all bits)
Console.WriteLine(h << 1); // Output: 12 (1100 binarnie)
Console.WriteLine(h >> 1); // Output: 3 (011 binarnie)
```


Operatory bitowe - przykład: Manipulacja kolorami RGB

A teraz bardziej złożony przykład wykorzystania operatorów bitowych w C#. W tym przykładzie stworzymy program, który będzie manipulował kolorami w formacie RGB, używając operatorów bitowych do ekstrakcji i modyfikacji poszczególnych składowych koloru.

W formacie RGB kolor jest reprezentowany przez trzy składowe: czerwoną (R), zieloną (G) i niebieską (B). Każda składowa jest 8-bitową liczbą całkowitą (0-255). Możemy przechowywać cały kolor w jednej 32-bitowej liczbie całkowitej, gdzie 8 najstarszych bitów jest nieużywanych, a pozostałe 24 bity są podzielone na trzy grupy po 8 bitów dla każdej składowej.

```
using System;
```

```
class Program
```

```
{
    static void Main()
    {
        // Przykładowy kolor: 0x00RRGGBB
        int color = 0x123456;

        // Ekstrakcja składowych koloru
        int red = (color >> 16) & 0xFF;
        int green = (color >> 8) & 0xFF;
        int blue = color & 0xFF;

        Console.WriteLine($"Original color: #{color:X6}");
        Console.WriteLine($"Red: {red}, Green: {green}, Blue: {blue}");

        // Zmiana składowej koloru
        red = 200;
        green = 100;
        blue = 50;

        // Składanie nowego koloru
        color = (red << 16) | (green << 8) | blue;

        Console.WriteLine($"Modified color: #{color:X6}");
        Console.WriteLine($"Red: {red}, Green: {green}, Blue: {blue}");
    }
}
```

Wyjaśnienie:

Ekstrakcja składowych koloru:

Aby wyodrębnić składową czerwoną, przesuwamy bity koloru o 16 miejsc w prawo i maskujemy je za pomocą 0xFF (bitowa maska 11111111).

Aby wyodrębnić składową zieloną, przesuwamy bity koloru o 8 miejsc w prawo i maskujemy je za pomocą 0xFF.

Aby wyodrębnić składową niebieską, maskujemy bity koloru za pomocą 0xFF.

Zmiana składowych koloru:

Przypisujemy nowe wartości do składowych czerwonej, zielonej i niebieskiej.

Składanie nowego koloru:

Przesuwamy bity składowej czerwonej o 16 miejsc w lewo, składowej zielonej o 8 miejsc w lewo i łączymy je za pomocą operatora bitowego OR (|) z wartością składowej niebieskiej.

Ten przykład pokazuje, jak można używać operatorów bitowych do manipulacji poszczególnymi składowymi koloru w formacie RGB. Jest to bardziej złożony przykład, który ilustruje praktyczne zastosowanie operatorów bitowych w programowaniu.

Instrukcja warunkowa if

W programowaniu, szczególnie w języku C#, często mamy do czynienia z sytuacjami, w których program musi podejmować decyzje w zależności od spełnienia określonych warunków. Do tego służą właśnie instrukcje warunkowe i wyboru. Dzięki nim możemy stworzyć bardziej dynamiczne i elastyczne aplikacje.

```
if (warunek)
{
    // Instrukcje wykonywane, jeśli warunek jest prawdziwy
}
else
{
    // Instrukcje wykonywane, jeśli warunek jest fałszywy
}
```

Warunek: Wyrażenie logiczne, które może być prawdziwe (true) lub fałszywe (false).

Bloki instrukcji: Zestawy instrukcji, które są wykonywane odpowiednio, gdy warunek jest spełniony lub nie.

```
int wiek = 25;

if (wiek >= 18)
{
    Console.WriteLine("Jesteś pełnoletni.");
}
else
{
    Console.WriteLine("Jesteś niepełnoletni.");
}
```

Instrukcja wyboru switch

```
switch (wyrażenie)
{
    case wartość1:
        // Instrukcje dla wartości1
        break;
    case wartość2:
        // Instrukcje dla wartości2
        break;
    default:
        // Instrukcje dla pozostałych wartości
}
```

Wyrażenie: Wartość, która jest porównywana z podanymi wartościami przypadków.

Case: Określa konkretną wartość, dla której mają być wykonane instrukcje.

Default: Blok instrukcji, który jest wykonywany, jeśli żadna z wartości case nie pasuje.

Break: Służy do wyjścia z bloku switch po wykonaniu odpowiednich instrukcji.

```

int dzienTygodnia = 3;

switch (dzienTygodnia)
{
    case 1:
        Console.WriteLine("Poniedziałek");
        break;
    case 2:
        Console.WriteLine("Wtorek");
        break;
    case 3:
        Console.WriteLine("Środa");
        break;
    default:
        Console.WriteLine("Nieznany dzień tygodnia");
        break;
}

```

Kiedy używać której instrukcji?

- if: Kiedy mamy do sprawdzenia jeden lub kilka warunków.
 Kiedy warunki są bardziej złożone i wymagają kombinacji operatorów logicznych.
- switch: Kiedy chcemy porównać jedną wartość z wieloma stałymi wartościami.
 Kiedy wartości są typu liczbowego lub znakowego.

Zagnieżdżanie instrukcji

Instrukcje warunkowe i wyboru można zagnieżdżać, tworząc bardziej złożone konstrukcje decyzyjne.

```

if (wiek >= 18)
{
    if (wiek >= 65)
    {
        Console.WriteLine("Jesteś emerytem.");
    }
    else
    {
        Console.WriteLine("Jesteś dorosły.");
    }
}
else
{
    Console.WriteLine("Jesteś niepełnoletni.");
}

```

Operator warunkowy ?:

Jest to skrócona forma instrukcji if, która może być używana w prostych przypadkach.

```

int wynik = liczba > 0 ? 1 : -1;

```

Instrukcje warunkowe i wyboru są fundamentalnymi narzędziami w programowaniu C#. Dzięki nim możemy tworzyć programy, które reagują na różne sytuacje i podejmują odpowiednie decyzje. Wybór odpowiedniej instrukcji zależy od konkretnego problemu, który chcemy rozwiązać.

ZADANIA

1. Operatory bitowe

Napisz program, który będzie manipulował kolorami w formacie RGB, używając operatorów bitowych do ekstrakcji i modyfikacji poszczególnych składowych koloru. Użyj następujących danych wejściowych:

Numer w dzienniku jako wartość początkowa koloru (w formacie szesnastkowym, np. 0x123456).

Liczba liter w imieniu jako nowa wartość składowej czerwonej.

Liczba liter w nazwisku jako nowa wartość składowej zielonej.

Suma liczby liter w imieniu i nazwisku jako nowa wartość składowej niebieskiej.

Przykład:

Jeśli Twój numer w dzienniku to 25, liczba liter w imieniu to Anna (4 litery), a liczba liter w nazwisku to Kowalska (8 liter), to:

Wartość początkowa koloru: 0x25 (w formacie szesnastkowym: 0x000025)

Nowa wartość składowej czerwonej: 4

Nowa wartość składowej zielonej: 8

Nowa wartość składowej niebieskiej: $4 + 8 = 12$

Następnie:

Wyodrębnij składowe koloru z wartości początkowej, używając operatorów bitowych.

Zmień składowe koloru na podstawie liczby liter w imieniu, nazwisku i ich sumy.

Złóż nowy kolor, przesuwając bity składowych i łącząc je za pomocą operatora bitowego OR (|).

2. Operacje na typach złożonych:

Napisz klasę Rectangle, która będzie miała właściwości Width i Height. Dodaj metodę CalculateArea, która zwróci pole prostokąta. Użyj numeru w dzienniku jako Width i liczby liter w imieniu jako Height. Oblicz pole prostokąta.

3. Operatory arytmetyczne:

Napisz program, który poprosi użytkownika o podanie dwóch liczb: numeru w dzienniku i liczby liter w nazwisku. Następnie wyświetli wynik dodawania, odejmowania, mnożenia, dzielenia i operacji modulo tych liczb.

4. Operatory przypisania:

Utwórz zmienną typu int i przypisz jej wartość równą liczbie liter w imieniu. Następnie użyj różnych operatorów przypisania (+, -, *, /, %) z numerem w dzienniku i wyświetl wynik po każdej operacji.

5. Operatory porównania:

Napisz program, który poprosi użytkownika o podanie dwóch liczb: numeru w dzienniku i liczby liter w nazwisku. Następnie wyświetli wyniki porównań (==, !=, <, >, <=, >=) tych liczb.

6. Operatory logiczne:

Napisz program, który poprosi użytkownika o podanie dwóch wartości logicznych (true lub false). Użyj numeru w dzienniku jako pierwszej wartości (true jeśli numer jest parzysty, false jeśli nieparzysty) i liczby liter w imieniu jako drugiej wartości (true jeśli liczba liter jest większa niż 5, false w przeciwnym razie). Następnie wyświetl wynik operacji logicznych (&&, ||, !) na tych wartościach.

7. Operatory bitowe:

Napisz program, który poprosi użytkownika o podanie dwóch liczb: numeru w dzienniku i liczby liter w nazwisku. Następnie wyświetl wynik operacji bitowych (&, |, ^, ~, <<, >>) na tych liczbach.

8. Operacje na typach złożonych:

Utwórz strukturę `ComplexNumber`, która będzie reprezentować liczby zespolone (z polami `Real` i `Imaginary`). Użyj numeru w dzienniku jako `Real` i liczby liter w imieniu jako `Imaginary`. Dodaj metody do dodawania, odejmowania, mnożenia i dzielenia liczb zespolonych. Przetestuj te metody na kilku przykładach.

9. Operatory arytmetyczne i przypisania:

Napisz program, który obliczy sumę, różnicę, iloczyn i iloraz dwóch liczb: numeru w dzienniku i liczby liter w nazwisku, a następnie przypisze te wyniki do odpowiednich zmiennych za pomocą operatorów przypisania.

10. Operatory porównania i logiczne:

Napisz program, który poprosi użytkownika o podanie trzech liczb: numeru w dzienniku, liczby liter w imieniu i liczby liter w nazwisku. Sprawdź, czy pierwsza liczba jest większa od drugiej i mniejsza od trzeciej, używając operatorów porównania i logicznych. Wyświetl odpowiedni komunikat.

11. Operatory bitowe:

Napisz program, który poprosi użytkownika o podanie liczby całkowitej (numeru w dzienniku). Następnie wyświetl wynik przesunięcia bitowego tej liczby w lewo i w prawo o liczbę pozycji równą liczbie liter w imieniu.

```
using System;

class Program
{
    static void Main()
    {
        // Dane wejściowe
        int journalNumber = 25; // Numer w dzienniku
        int nameLength = 4;    // Liczba liter w imieniu
        int surNameLength = 8; // Liczba liter w nazwisku
        int totalLength = nameLength + surNameLength;

        // Wartość początkowa koloru
        int color = journalNumber;

        // Ekstrakcja składowych koloru
        int red = (color >> 16) & 0xFF;
        int green = (color >> 8) & 0xFF;
        int blue = color & 0xFF;

        Console.WriteLine($"Original color: #{color:X6}");
        Console.WriteLine($"Red: {red}, Green: {green}, Blue: {blue}");

        // Zmiana składowych koloru
        red = nameLength;
        green = surNameLength;
        blue = totalLength;

        // Składanie nowego koloru
        color = (red << 16) | (green << 8) | blue;

        Console.WriteLine($"Modified color: #{color:X6}");
        Console.WriteLine($"Red: {red}, Green: {green}, Blue: {blue}");
    }
}
```